DOSSIER TECHNIQUE

PRÉDICTION DE LA NOTE À PARTIR DES TRACES SUR ARCHE

Par Khady DIAGNE
IDMC | UNIVERSITE DE LORRAINE 2025 – 2026
GITHUB : HTTPS://GITHUB.COM/LADIJADI/PROJET_ARCHE

Table des matières

Table des matières 1
Introduction 3
1. Problématique 3
2. Architecture du système 3
2.1. Organisation du projet 3
2.2. Description des modules 4
2.3. Diagramme de classes (UML) 5
2.4. Conception du pipeline 5
2.5. Diagramme de séquence 6
3. Module de chargement des données 7
4. Module de prétraitement des données 7
4.1. Traitement du fichier des notes 8
4.2. Traitement du fichier des logs 8
4.3. Catégorisation des événements dans logs 9
4.4. Mise en cohérence des deux sources de donnée : logs et notes 9
4.5. Audit complémentaire de qualité 10
4.6. Les Algorithmes de prétraitement des données 11
4.7. Diagramme activité du module Prétraitement 13
5. Module de construction des variables explicatives 15
5.1. Variables structurelles 15
5.2. Variable temporelle 15
5.3. Variables comportementales 16
5.4. Variable liée au composant 16
5.5. Fusion finale 16
5.6. Algorithmes : Feature Engineering 16
5.7. Diagramme d'activité Feature Engineering 20
6. Module de régression multiple 21
6.1. Sélection des variables 21

6.2. Estimation de la régression multiple 21
6.3. Contrôle de multicolinéarité 22
6.4. Algorithmes de la Régression Multiple 22
6.5. Diagramme d'activité 24
7. Module modèle comparatif : Arbre de décision régressif 25
7.1. Définition des variables explicatives 25
7.2. Entraînement du modèle 25
7.3. Évaluation des performances 25
7.4. Interprétation du modèle 26
7.5. Algorithmes de l'arbre de décision régressif 26
7.6. Diagramme d'activité de l'arbre de décision régressif 29
8. Module d'évaluation comparative 30
8.1. Calcul des métriques 30
8.2. Comparaison des modèles 30
8.3. Visualisation des performances 30
8.4. Sorties du module 31
8.5. Algorithmes de l'évaluation Comparative 31
8.6. Diagramme d'activité de l'évaluation comparative des modèles 34
9. Module Interface Utilisateur 35
9.1. Organisation générale 35
9.2. Initialisation et chargement des modèles 35
9.3. Saisie dynamique des variables 36
9.4. Validation des entrées 36
9.5. Production et affichage des résultats 36
9.6. Intérêt du module 37
9.7. Algorithmes de l'interface utilisateur 37
9.8. Diagramme d'activité de l'interface 40
Conclusion générale 41

Introduction

Ce projet s'inscrit dans le cadre d'un travail commun aux enseignements d'algorithmique, de programmation et de mathématiques pour l'informatique. L'objectif est de proposer un modèle capable de prédire la note d'un étudiant à partir de ses traces d'activité sur la plateforme pédagogique ARCHE.

Le projet repose sur deux sources de données :

- un fichier *logs.csv* contenant les traces d'usage de la plateforme,
- un fichier *notes.csv* contenant la note finale de chaque étudiant.

Deux approches de modélisation ont été comparées :

1) une régression linéaire multiple imposée
2) un arbre de décision régressif pour comparer

Une interface graphique Tkinter a également été développée afin de rendre le modèle exploitable par un utilisateur non technique.

1. Problématique

Est-il possible de prédire la note d'un apprenant à partir de ses traces numériques au sein de la plateforme ARCHE ?

L'objectif est également d'identifier les comportements associés à la réussite ou à l'échec.

2. Architecture du système

2.1. Organisation du projet

L'application est structurée en plusieurs modules correspondant aux différentes étapes du pipeline de traitement des données.

```
Projet_Arche/
data/
    logs.csv
    notes.csv
src/
    config.py
    data_loader.py
    exploration.py
    preprocessing.py
    features_engineering.py
```

```
        multiple_regression.py
        comparison_model.py
        evaluation.py
        interface.py
    output/
    main.py
```

2.2. Description des modules

config.py : centralise les constantes du projet

data_loader.py : charge les fichiers et vérifie leur conformité

preprocessing.py : nettoie et prépare les données (dates, types, filtrage des étudiants)

exploration.py : réalise l'exploration descriptive et graphique

features_engineering.py : construit les variables explicatives utilisés pour la modélisation

multiple_regression.py : implémente la sélection de variables, entraîne la régression multiple et calcule la VIF (Variance Inflation Factor) afin de détecter d'éventuelles situations de multicolinéarité entre les variables explicatives.

comparison_model.py : implémente un modèle alternatif basé sur un arbre de décision

evaluation.py : compare les performances des modèles

interface.py : gère l'interface utilisateur et l'interaction avec les modèles

output.py : contient les images sauvegardées

main.py : point d'entrée du projet

2.3. Diagramme de classes (UML)

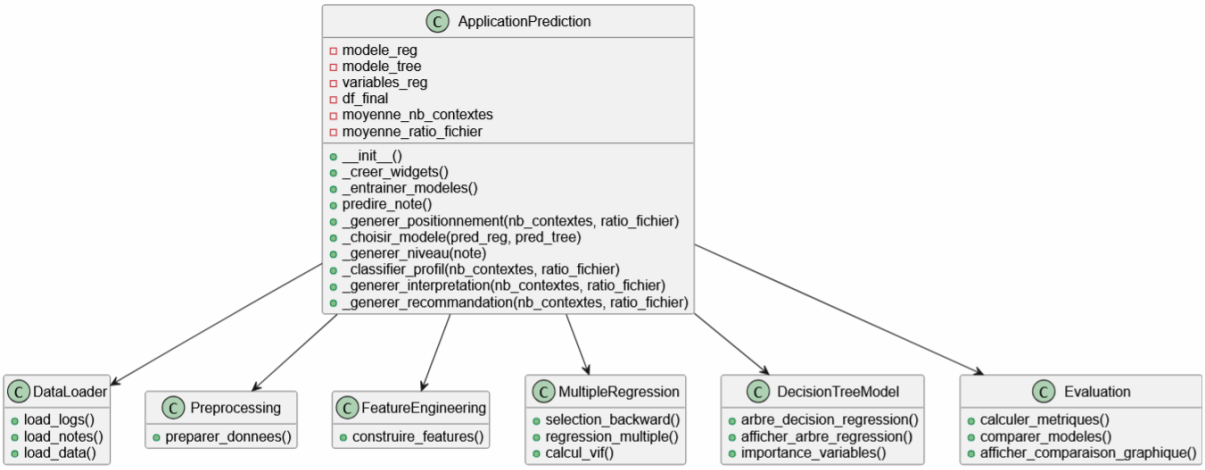


Figure 1 : Diagramme de classes de l'application

Le diagramme de classes présente l'organisation logique du logiciel. La classe principale *ApplicationPrediction*, développée avec Tkinter, constitue le point d'entrée de l'interface utilisateur. Elle s'appuie sur plusieurs modules spécialisés : chargement des données, prétraitement, construction des variables, modélisation et comparaison des performances. Cette organisation met en évidence une séparation claire des responsabilités, ce qui améliore la lisibilité, la maintenabilité et la réutilisabilité du code.

2.4. Conception du pipeline

logs.csv + notes.csv

↓

Chargement sécurisé

↓

Prétraitement

↓

Exploration descriptive

↓

Construction des features

↓

Régression multiple + arbre de décision

↓

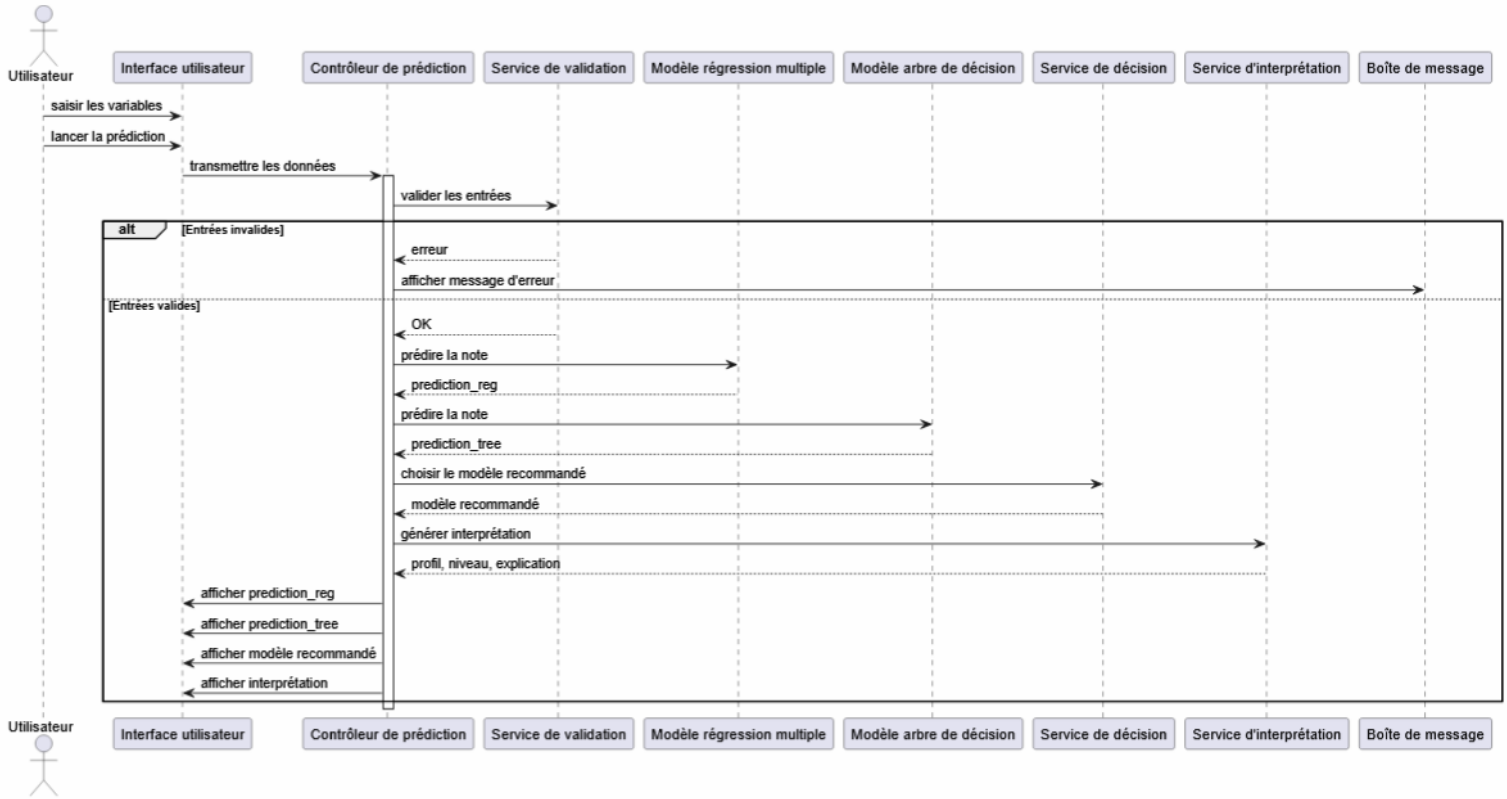
Évaluation comparative

↓

Interface utilisateur

Cette organisation suit une logique de traitement séquentiel et modulaire, facilitant la compréhension et la maintenance du système.

2.5. Diagramme de séquence



Figue 2 : Diagramme de Séquence

Ce diagramme de séquence illustre le déroulement complet du processus de prédiction à partir d'une saisie utilisateur.

L'utilisateur renseigne les variables d'entrée dans l'interface, qui transmet la requête au contrôleur de prédiction. Les données sont d'abord validées afin de garantir leur cohérence. En cas d'erreur, un message est affiché à l'utilisateur.

Si les entrées sont valides, le contrôleur sollicite successivement les deux modèles (régression multiple et arbre de décision) afin de produire deux estimations de la note.

Un service de décision est ensuite utilisé pour déterminer le modèle recommandé, en s'appuyant sur les performances globales des modèles évaluées en phase d'apprentissage.

Enfin, un service d'interprétation génère une explication du résultat (profil, niveau, recommandations), et l'ensemble des résultats est affiché dans l'interface utilisateur.

3. Module de chargement des données

Le module data_loader.py assure le chargement sécurisé des deux fichiers.

Contrôles réalisés :

- ✓ Existence des fichiers ;
- ✓ Lecture des fichiers CSV ;
- ✓ Détection des fichiers vides ;
- ✓ Contrôle des colonnes attendues ;
- ✓ Gestion des erreurs de lecture.

Colonnes attendues

Pour logs.csv :

- ✓ heure
- ✓ pseudo
- ✓ contexte
- ✓ composant
- ✓ evenement

Pour notes.csv :

- ✓ pseudo
- ✓ note

Cette étape garantit que le reste du pipeline ne s'exécute que sur des données structurellement valides.

4. Module de prétraitement des données

Le module *preprocessing.py* a pour rôle de transformer les données brutes issues du chargement en données cohérentes, exploitables et compatibles avec les étapes ultérieures du projet. Il intervient immédiatement après le chargement des fichiers et avant l'exploration, la construction des variables et la modélisation.

Le module preprocessing.py nettoie les deux fichiers avant l'analyse.

Le prétraitement vise à transformer les données brutes en données cohérentes, exploitables et compatibles avec la modélisation. Il repose sur quatre traitements principaux : le nettoyage du fichier des notes, le nettoyage du fichier des logs, la catégorisation des événements et la mise en cohérence entre les deux sources. Un audit complémentaire de qualité est ensuite réalisé afin de vérifier la validité structurelle des données obtenues.

4.1. Traitement du fichier des notes

Le fichier notes est d'abord recopié afin d'éviter toute modification directe des données d'origine. Le champ pseudo est ensuite converti en numérique, ce qui permet d'éliminer les identifiants invalides. Les doublons exacts sont supprimés. Enfin, la variable cible note est convertie en numérique, puis les lignes contenant des valeurs non exploitables sont retirées.

Les opérations réalisées sont :

- ✓ Conversion de pseudo en numérique
- ✓ Suppression des valeurs invalides
- ✓ Suppression des doublons exacts
- ✓ Conversion de la note en numérique
- ✓ Suppression des notes invalides

Résultats obtenus

Le fichier notes.csv est apparu globalement propre :

- ✓ Aucun pseudo invalide
- ✓ Aucun doublon
- ✓ Aucune note invalide
- ✓ 99 étudiants au final

4.2. Traitement du fichier des logs

Le fichier logs est également recopié avant traitement. Le champ pseudo est converti en numérique, puis le champ heure est transformé en objet temporel. Les colonnes textuelles (contexte, composant, evenement) sont normalisées par suppression des espaces superflus, passage en minuscules et suppression des accents. Les lignes entièrement vides sur ces dimensions sont écartées, ainsi que les doublons exacts.

Les opérations réalisées sont :

- ✓ Conversion de pseudo en numérique
- ✓ Conversion de heure au format datetime
- ✓ Normalisation des champs texte avec :
    - Suppression des accents,
    - Passage en minuscules,
    - Suppression des espaces inutiles ;
    - Suppression des lignes totalement vides ;
    - Suppression des doublons exacts.

Résultats obtenus :

Le fichier logs.csv était globalement exploitable :

- ✓ Aucun pseudo invalide ;
- ✓ Aucune date invalide ;
- ✓ Aucune ligne vide ;
- ✓ 28 doublons supprimés ;
- ✓ 16199 lignes conservées après nettoyage.

4.3. Catégorisation des événements dans logs

Une catégorisation des événements est ensuite créée afin de regrouper les libellés détaillés en grandes classes comportementales (test, consultation, interaction, navigation, visite_guidee, autre).

Résultats :

Afin de réduire la granularité des événements et de rendre leur interprétation plus simple, une catégorisation a été introduite :

- Consultation
- Test
- Interaction
- Navigation
- visite_guidee
- Autre

Cette étape permet de passer de libellés textuels très détaillés à des catégories comportementales plus pertinentes pour l'analyse.

4.4. Mise en cohérence des deux sources de donnée : logs et notes

Une fois les deux fichiers nettoyés, les logs sont filtrés pour ne conserver que les étudiants présents dans le fichier des notes. Cette étape garantit la construction d'un jeu supervisé cohérent. Le module identifie également les étudiants présents dans notes mais absents des logs, ce qui permet de repérer les cas sans activité enregistrée.

Résultats :

- ✓ 109 étudiants présents dans les logs avant filtrage ;
- ✓ 99 étudiants présents dans les notes ;
- ✓ 95 étudiants conservés dans les logs après filtrage ;
- ✓ 4 étudiants présents dans notes.csv mais sans activité ARCHE.

Limite :

L'exclusion des étudiants sans activité peut introduire un biais, car ces profils sont potentiellement les moins engagés.

4.5. Audit complémentaire de qualité

Le module inclut enfin une fonction d'audit permettant de vérifier :

l'absence de valeurs manquantes résiduelles ;

la validité des notes ;

la validité des identifiants ;

la cohérence temporelle des logs.

Ce contrôle ne modifie pas les données, mais documente leur qualité après prétraitement.

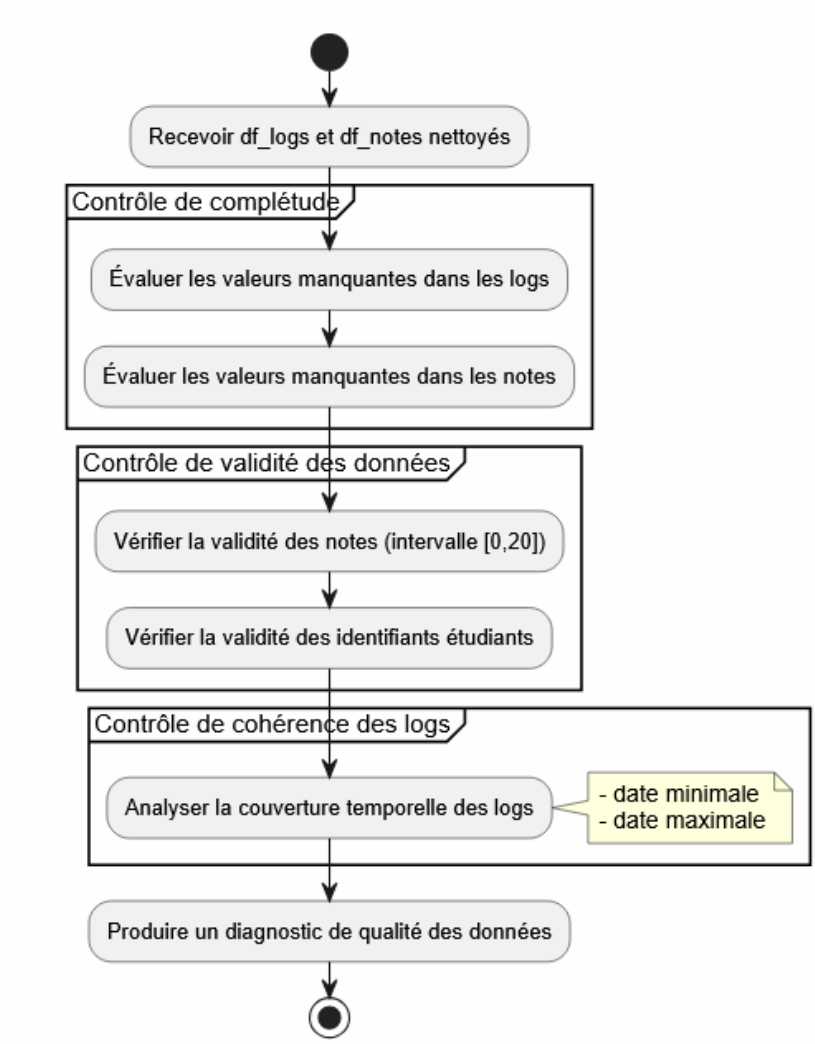


Figure 3 : Diagramme d'activité de l'audit qualité

Ce diagramme présente les contrôles complémentaires réalisés après le prétraitement afin de vérifier la validité structurelle des données obtenues.

4.6. Les Algorithmes de prétraitement des données

4.6.1. Algorithme Principale : preparer_donnees(df_logs, df_notes)

Entrées :

- df_logs : données brutes des logs
- df_notes : données brutes des notes

Variables :

- df_logs_traite : logs nettoyés et transformés
- df_notes_traite : notes nettoyées
- df_logs_filtre : logs restreints aux étudiants présents dans les notes

Sorties :

- df_logs_filtre : logs prétraités
- df_notes_traite : notes prétraitées

```
DEBUT
  SI df_logs est Nul OU df_notes est Nul ALORS
    Retourner (Nul, Nul)
  SINON
    df_notes_traite <- traiter_notes(df_notes)
    df_logs_traite <- traiter_logs(df_logs)
    df_logs_filtre <- filtrer_etudiants(df_logs_traite, df_notes_traite)

    Retourner (df_logs_filtre, df_notes_traite)
  FINSI
FIN
```

4.6.2. Sous-algorithme : traiter_notes(df_notes)

```
DEBUT
  Convertir pseudo en numérique
  Supprimer les pseudos invalides
  Convertir pseudo en entier
  Supprimer les doublons exacts
  Convertir note en numérique
```

Supprimer les notes invalides
Retourner df_notes nettoyé
FIN

4.6.3. Sous-algorithme : traiter_logs(df_logs)

DEBUT
Convertir pseudo en numérique
Supprimer les pseudos invalides
Convertir pseudo en entier
Convertir heure en date-heure
Supprimer les dates invalides
POUR chaque colonne parmi contexte, composant, evenement
Remplacer les valeurs manquantes par ""
Normaliser le texte
FAIT
Supprimer les lignes telles que
contexte = "" ET composant = "" ET evenement = ""
Supprimer les doublons exacts
POUR chaque evenement
Calculer categorie_evenement
FAIT
Retourner df_logs nettoyé
FIN

4.6.4. Sous-algorithme : categoriser_evenement(evenement)

DEBUT
SI "test" appartient à evenement alors
Retourner "test"
SINONSI "cours" appartient à evenement OU "module" appartient à evenement alors
Retourner "consultation"
SINONSI "discussion" appartient à evenement OU "contenu poste" appartient à evenement alors
Retourner "interaction"
SINONSI "visite guidee" appartient à evenement alors
Retourner "visite_guidee"
SINONSI "profil utilisateur" appartient à evenement OU "rapport" appartient à evenement alors
Retourner "navigation"
SINON

Retourner "autre"
FINSI
FIN

4.6.5. Sous-algorithme : filtrer_etudiants(df_logs, df_notes)

DEBUT
Extraire l'ensemble des étudiants présents dans notes
Extraire l'ensemble des étudiants présents dans logs
Conserver dans logs uniquement les étudiants présents dans notes
Identifier les étudiants présents dans notes mais absents de logs
Retourner df_logs filtré
FIN

4.6.6. Sous-algorithme : audit_qualite_donnees(df_logs, df_notes)

DEBUT
Afficher les valeurs manquantes dans logs
Afficher les valeurs manquantes dans notes
Compter les notes hors de l'intervalle [0,20]
Compter les pseudos <= 0 dans notes
Compter les pseudos <= 0 dans logs
Afficher la date minimale des logs
Afficher la date maximale des logs
FIN

4.7. Diagramme activité du module Prétraitement

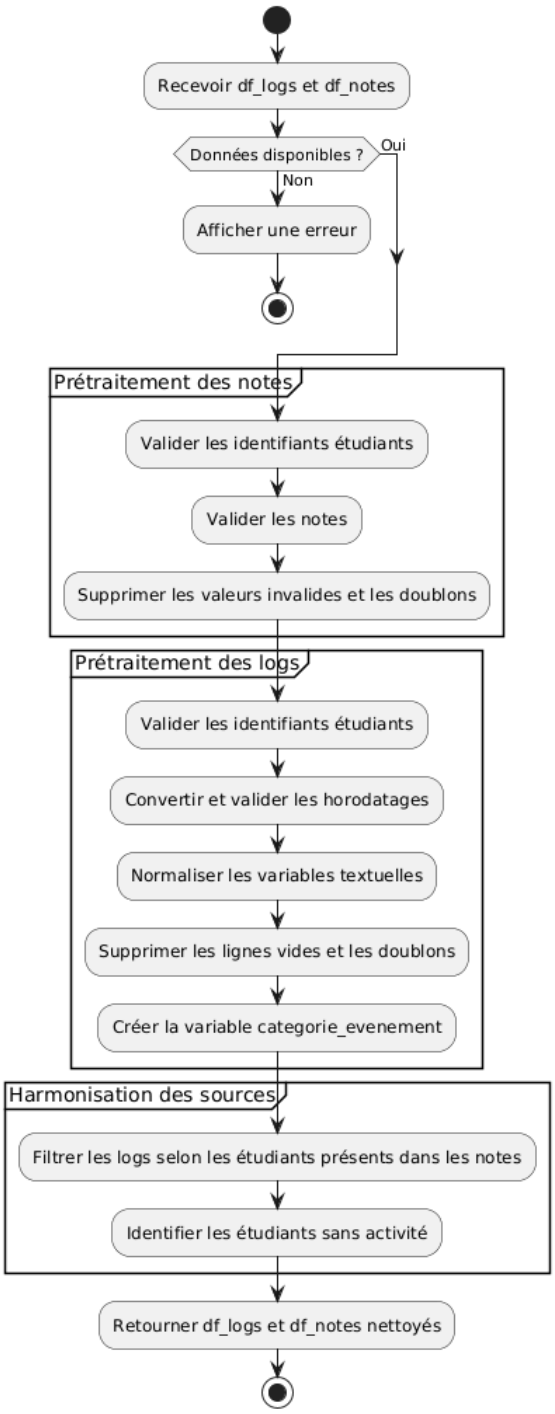


Figure 4 : Diagramme d'activité du prétraitement des données

Ce diagramme décrit la chaîne de nettoyage appliquée aux fichiers logs et notes, depuis la vérification de disponibilité des données jusqu'à la production de jeux de données cohérents pour la suite du pipeline.

5. Module de construction des variables explicatives

Le module **features_engineering.py** transforme les logs prétraités en un jeu de données agrégé au niveau étudiant, utilisable par les modèles de prédiction.

L'objectif de cette étape est de passer d'un ensemble de traces unitaires, enregistrées action par action, à un ensemble de variables explicatives synthétiques, où chaque ligne représente un étudiant. Le module construit plusieurs familles d'indicateurs.

5.1. Variables structurelles

Les premières variables décrivent l'intensité et la diversité globale de l'activité :

nb_contextes : nombre de contextes distincts consultés ;

nb_actions : nombre total d'actions ;

nb_jours_actifs : nombre de jours distincts d'activité.

Ces variables permettent de représenter le volume d'usage, la diversité des ressources consultées et la régularité de l'activité.

5.2. Variable temporelle

Le module construit ensuite deux variables temporelles destinées à approcher l'intensité d'activité sur la plateforme.

La première, *temps_moyen_action*, correspond à une estimation du temps moyen d'activité par action. Pour la calculer :

- ✓ les logs sont triés par étudiant et par date,
- ✓ l'écart entre deux actions successives est calculé,
- ✓ seuls les écarts strictement positifs et inférieurs à 300 secondes sont conservés,
- ✓ le temps total estimé est ensuite rapporté au nombre total d'actions.

La seconde, *temps_moyen_jour_actif*, vise à mesurer l'intensité moyenne de l'activité par jour actif. Pour cela :

- ✓ les logs sont regroupés par étudiant et par jour ;
- ✓ les écarts entre actions successives sont calculés au sein d'une même journée ;
- ✓ seuls les écarts strictement positifs et inférieurs à 300 secondes sont conservés ;
- ✓ le temps total estimé est calculé pour chaque jour actif ;
- ✓ la moyenne de ces temps journaliers est ensuite calculée par étudiant.

Cette règle permet d'écarter les interruptions longues, qui ne correspondent pas à une activité continue sur la plateforme, et d'introduire une lecture temporelle plus fine du comportement étudiant.

5.3. Variables comportementales

Le module exploite ensuite la variable categorie_evenement construite lors du prétraitement.

Pour chaque étudiant, il calcule :

- ratio_test
- ratio_interaction
- ratio_consultation
- engagement_actif

L'indicateur engagement_actif synthétise la part des comportements considérés comme actifs, en combinant les catégories test et interaction.

5.4. Variable liée au composant

Enfin, le module calcule le ratio_fichier qui est la part des actions dont le composant est fichier. Elle permet de quantifier une forme d'usage centrée sur les ressources de type fichier.

5.5. Fusion finale

Toutes les variables construites sont fusionnées avec le fichier des notes, afin d'obtenir un jeu de données final df_final. Les valeurs manquantes résiduelles sont remplacées par 0, ce qui permet de conserver un tableau complet pour la modélisation.

5.6. Algorithmes : Feature Engineering

La construction des variables explicatives vise à résumer les comportements observés dans les logs sous forme d'indicateurs numériques au niveau étudiant.

Cette étape transforme les traces unitaires de navigation en un tableau structuré, directement exploitable par les modèles de prédiction.

5.6.1. Algorithme principale : construire_features(df_logs, df_notes)

Entrées :

- df_logs : logs nettoyés
- df_notes : notes nettoyées

Variables :

- data : copie de df_logs
- df_final : dataset final

- features : ensemble des tables de variables construites

Sortie :
- df_final : dataset contenant les features par étudiant

DEBUT
Copier df_logs dans data

Construire les variables structurelles :
- nb_contextes
- nb_actions
- nb_jours_actifs

Construire les variables temporelles :
- trier les logs par étudiant et par date
- calculer les écarts de temps entre actions successives
- conserver les écarts strictement positifs et inférieurs à 300 secondes
- calculer le temps moyen par action
- calculer le temps moyen par jour actif

Construire les variables comportementales à partir de categorie_evenement

Construire ratio_fichier à partir du composant "fichier"

Copier df_notes dans df_final

POUR chaque table de features construite
Fusionner la table avec df_final
FINPOUR

Remplacer les valeurs manquantes par 0

Retourner df_final
FIN

5.6.2. Sous-algorithme : calculer_variables_structurelles(df_logs)

DEBUT
Calculer le nombre de contextes distincts par étudiant
Calculer le nombre total d'actions par étudiant
Extraire le jour de chaque action
Calculer le nombre de jours actifs par étudiant
Calculer le temps moyen jours actif
Retourner nb_contextes, nb_actions, nb_jours_actifs
FIN

5.6.3. Sous-algorithme : calculer_temps_moyen_action(df_logs, nb_actions)

```
DEBUT
  Trier df_logs par pseudo puis par heure
  Calculer l'écart entre deux actions successives
  Conserver uniquement les écarts > 0 et < 300 secondes
  Calculer le temps total par étudiant
  POUR chaque étudiant
    SI nb_actions > 0 alors
      temps_moyen_action ← temps_total / nb_actions
    SINON
      temps_moyen_action ← 0
    FINSI
  FAIT
  Retourner temps_moyen_action
FIN
```

5.6.4. Sous-algorithme calcul_temps_moyen_jour_actif(df_logs)

```
DEBUT
  Copier le dataframe des logs

  Trier les données par étudiant et par date

  Créer une variable "jour" à partir de la date

  Calculer l'écart de temps entre deux actions successives
  pour chaque étudiant et pour chaque jour

  Conserver uniquement les écarts strictement positifs
  et inférieurs à 300 secondes

  Calculer le temps total d'activité par étudiant et par jour
  en sommant les écarts retenus

  Calculer le temps moyen d'activité par jour actif
  pour chaque étudiant

  Retourner un dataframe contenant :
    l'identifiant de l'étudiant
    le temps moyen par jour actif
FIN
```

5.6.5. Sous-algorithme : calculer_ratios_comportementaux(df_logs, nb_actions)

```
DEBUT
```

Compter les actions par categorie_evenement et par étudiant
SI la colonne test n'existe pas alors
Ajouter la colonne test et l'initialiser à 0
FINSI
SI la colonne interaction n'existe pas alors
Ajouter la colonne interaction et l'initialiser à 0
FINSI
SI la colonne consultation n'existe pas alors
Ajouter la colonne consultation et l'initialiser à 0
FINSI
Fusionner avec nb_actions
Calculer ratio_test
Calculer ratio_interaction
Calculer ratio_consultation
Calculer engagement_actif = (test + interaction) / nb_actions
Retourner les ratios comportementaux
FIN

5.6.6. Sous-algorithme : calculer_ratio_fichier(df_logs, nb_actions)

DEBUT
Sélectionner les actions dont le composant est "fichier"
Compter le nombre d'actions fichier par étudiant
Fusionner avec nb_actions
Calculer ratio_fichier = nb_fichier / nb_actions
Retourner ratio_fichier
FIN

5.7. Diagramme d'activité Feature Engineering

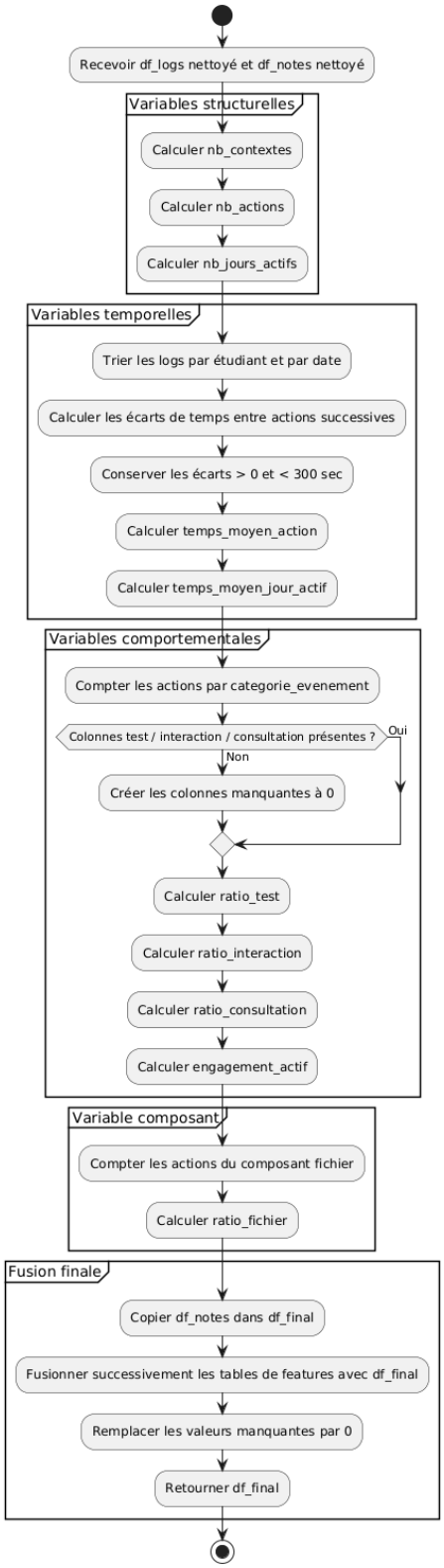


Figure 6 : Diagramme d'activité de la Feature Engineering

Ce diagramme décrit la transformation des logs prétraités en un jeu de variables agrégées par étudiant. Il met en évidence la construction de variables structurelles, temporelles, comportementales et liées aux composants.

6. Module de régression multiple

Le module multiple_regression.py met en œuvre la modélisation linéaire du problème de prédiction de la note.

Il est organisé en trois traitements :

- ✓ une **sélection descendante des variables** par significativité statistique ;
- ✓ une **régression linéaire multiple** évaluée sur un découpage apprentissage/test ;
- ✓ un **contrôle de multicolinéarité** à l'aide du VIF.

6.1. Sélection des variables

Le module commence par ajuster un modèle OLS sur un ensemble initial de variables candidates :

- nb_contextes
- ratio_test
- ratio_interaction
- temps_moyen_action
- temps_moyen_jour_actif
- ratio_fichier

Une procédure de sélection backward est appliquée. À chaque itération, la variable présentant la plus grande p-value est supprimée tant qu'elle reste supérieure au seuil fixé.
Cette stratégie permet de conserver uniquement les variables significatives dans le modèle final.

6.2. Estimation de la régression multiple

Une fois les variables retenues identifiées, une régression linéaire multiple est entraînée avec LinearRegression.

Les données sont séparées en deux sous-ensembles :

- un ensemble d'apprentissage ;
- un ensemble de test.

Le modèle est ensuite évalué à l'aide des métriques suivantes :

- ✓ MSE

- ✓ RMSE
- ✓ R^2

Les coefficients sont affichés afin de permettre une interprétation du sens de l'effet de chaque variable.

6.3. Contrôle de multicolinéarité

Le module calcule enfin le facteur d'inflation de variance (VIF) pour les variables retenues.

Ce contrôle permet de vérifier qu'aucune redondance excessive ne subsiste entre les variables explicatives du modèle final.

6.4. Algorithmes de la Régression Multiple

La régression multiple a pour objectif d'estimer la note à partir d'un ensemble restreint de variables comportementales. Le module combine une sélection statistique des variables, une estimation linéaire sur échantillon d'apprentissage/test et un contrôle de multicolinéarité afin d'obtenir un modèle à la fois interprétable et cohérent.

6.4.1. Algorithme principal : regression_multiple_complete(df_final)

Entrée :

- df_final : dataset final contenant la variable cible et les variables explicatives

Variables :

- variables_retenues : liste des variables sélectionnées
- modele_stats : modèle OLS final issu de la sélection backward
- modele : modèle de régression linéaire entraîné
- vif_data : tableau des facteurs d'inflation de variance

Sorties :

- modele : modèle de régression multiple estimé
- modele_stats : modèle statistique final issu de la sélection
- vif_data : tableau des VIF des variables retenues

DEBUT

Sélectionner les variables retenues par la procédure backward
Estimer la régression multiple sur les variables retenues
Évaluer le modèle sur l'échantillon de test
Calculer le VIF des variables retenues

Retourner le modèle estimé, le modèle statistique final et le tableau des VIF

FIN

6.4.2. Sous-algorithme : selection_backward(df_final)

DEBUT

Initialiser la liste des variables candidates

REPETER

Ajuster un modèle OLS avec les variables courantes

Calculer les p-values des variables

Identifier la variable ayant la p-value la plus élevée

SI cette p-value est supérieure au seuil

Supprimer la variable correspondante

SINON

Arrêter la sélection

FINSI

JUSQU'A obtention d'un ensemble de variables significatives

Retourner la liste des variables retenues et le modèle OLS final

FIN

6.4.3. Algorithme regression_multiple(df_final, variables_retenues)

DEBUT

Construire X à partir des variables retenues
Construire y à partir de la note

Découper les données en échantillon d'apprentissage et de test
Entraîner le modèle de régression linéaire sur l'échantillon d'apprentissage
Prédire les notes sur l'échantillon de test

Calculer MSE
Calculer RMSE
Calculer R^2

Retourner le modèle, y_test, y_pred, rmse et r2

FIN

6.4.4. Sous-algorithme calcul_vif(df_final, variables_retenues)

DEBUT

Construire la matrice des variables explicatives
Ajouter la constante
Calculer le VIF de chaque variable
Retourner le tableau des VIF

FIN

6.5. Diagramme d'activité

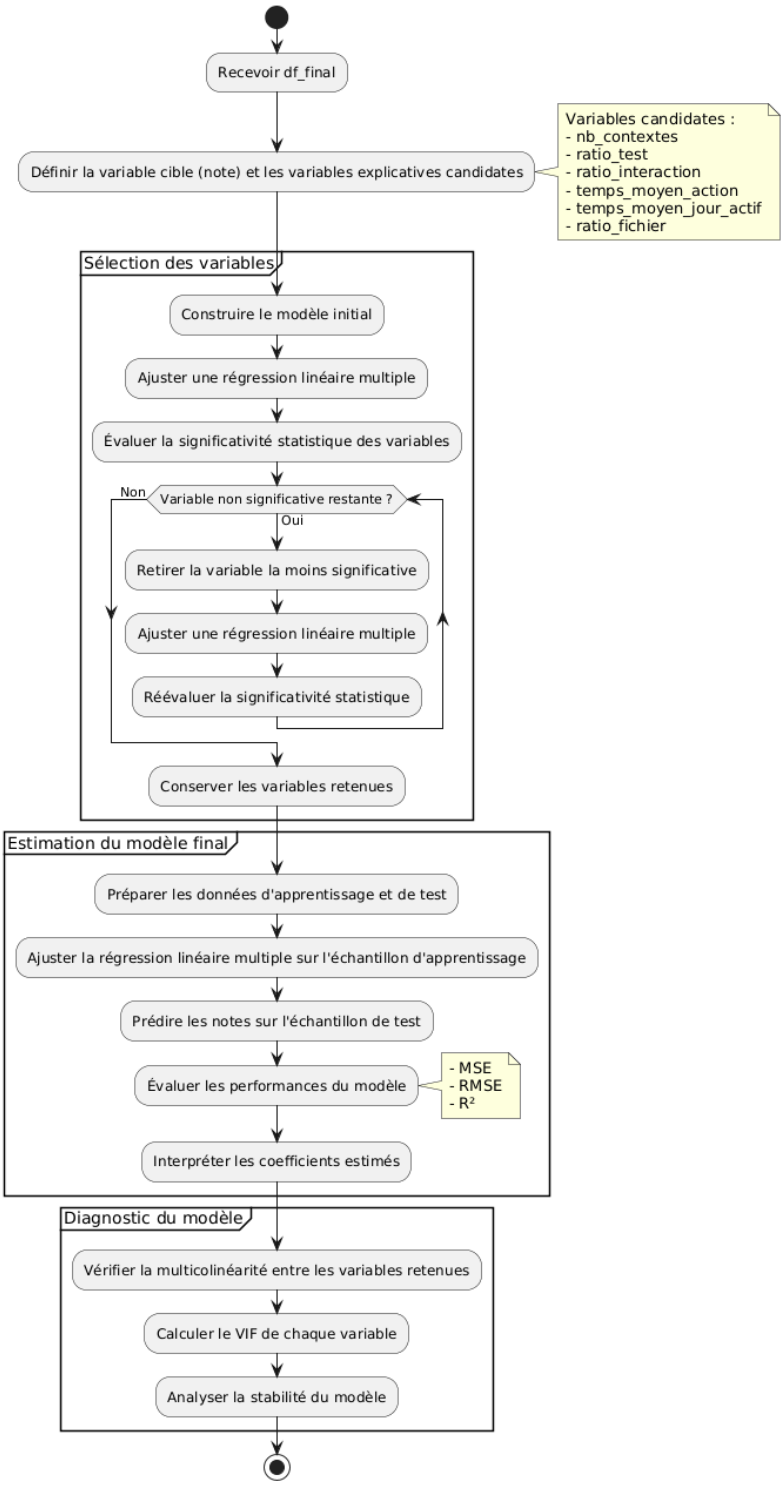


Figure 7 : Diagramme d'activité de la Régression Multiple

7. Module modèle comparatif : Arbre de décision régressif

Le module *comparison_model.py* met en œuvre un modèle d'arbre de décision régressif dans le cadre de la comparaison avec la régression multiple.

Son objectif est d'introduire une méthode de modélisation non linéaire, afin d'évaluer son impact sur la qualité de prédiction de la note.

- ✓ Le module est structuré en trois étapes :
- ✓ la définition et la vérification des variables explicatives
- ✓ l'entraînement du modèle sur un découpage apprentissage/test
- ✓ et l'évaluation des performances et l'interprétation du modèle.

7.1. Définition des variables explicatives

Le modèle utilise les variables issues de la sélection réalisée dans la régression multiple.

Ces variables sont transmises au module ou, à défaut, une liste de variables candidates est utilisée. Un contrôle est effectué afin de vérifier la présence des variables dans le dataframe.

7.2. Entraînement du modèle

Le module construit la matrice des variables explicatives X et la variable cible y.

Les données sont ensuite divisées en deux sous-ensembles : un ensemble d'apprentissage et un ensemble de test.

Le modèle utilisé est un DecisionTreeRegressor et deux paramètres principaux permettent de contrôler la complexité :

- max_depth : profondeur maximale de l'arbre ;
- et min_samples_leaf : nombre minimal d'observations par feuille.

Le modèle est entraîné sur l'échantillon d'apprentissage, puis utilisé pour prédire les valeurs sur l'échantillon de test.

7.3. Évaluation des performances

Les performances du modèle sont évaluées à l'aide des métriques suivantes :

- ✓ MSE
- ✓ RMSE
- ✓ MAE
- ✓ R^2.

Ces indicateurs permettent de mesurer l'erreur de prédiction et la qualité d'ajustement du modèle. Les résultats sont ensuite transmis au module d'évaluation comparative.

7.4. Interprétation du modèle

Le module propose deux outils d'interprétation :

- l'affichage de la structure de l'arbre de décision, permettant de visualiser les règles de segmentation ;
- le calcul de l'importance des variables, indiquant la contribution de chaque variable dans le modèle.

Ces éléments permettent d'analyser le comportement du modèle au-delà des seules métriques de performance.

7.5. Algorithmes de l'arbre de décision régressif

7.5.1. Algorithme principal arbre_decision_regression(df, variables, target, max_depth, min_samples_leaf)

Entrées :

- df_final : dataframe final contenant la variable cible et les variables explicatives
- variables : liste des variables explicatives à utiliser (optionnelle)

Variables :

- variables_utilisees : liste des variables effectivement retenues
- X : matrice des variables explicatives
- y : variable cible
- X_train, X_test : échantillons d'apprentissage et de test
- y_train, y_test : valeurs réelles d'apprentissage et de test
- y_pred : valeurs prédites
- modele : arbre de décision régressif
- mse, rmse, mae, r2 : métriques d'évaluation

Sorties :

- modele : modèle entraîné
- X_test : variables explicatives de l'échantillon de test
- y_test : valeurs réelles de l'échantillon de test
- y_pred : valeurs prédites
- rmse : racine de l'erreur quadratique moyenne
- r2 : coefficient de détermination
- variables_utilisees : variables effectivement utilisées

DEBUT
SI aucune variable n'est fournie **ALORS**
Charger la liste des variables candidates par défaut
FINSI

Vérifier que les variables existent dans df_final
Conserver uniquement les variables valides dans variables_utilisees

Construire X à partir de variables_utilisees
Construire y à partir de la variable cible

Découper les données en échantillon d'apprentissage et de test

Initialiser un arbre de décision régressif
Entraîner le modèle sur l'échantillon d'apprentissage

Prédire les notes sur l'échantillon de test

Calculer MSE
Calculer RMSE
Calculer MAE
Calculer R^2

Afficher les métriques

Retourner modele, X_test, y_test, y_pred, rmse, mae, r2, variables_utilisees
FIN

7.5.2. Sous-algorithme get_variables_comparaison

DEBUT
Retourner la liste des variables candidates :
nb_contextes
ratio_test
ratio_interaction
temps_moyen_action
temps_moyen_jour_actif
ratio_fichier
FIN

7.5.3. Sous-algorithme verifier_variables(df, variables)

DEBUT
Construire la liste des variables présentes dans le dataframe

SI aucune variable n'est valide **ALORS**

Déclencher une erreur
FINSI

Retourner la liste des variables présentes
FIN

7.5.4. Sous-algorithme afficher_arbre_regression(modele, variables)

DEBUT
Construire la figure graphique
Tracer l'arbre avec les noms des variables
Ajouter le titre
Afficher la figure
FIN

7.5.5. Algorithme importance_variables(modele, variables)

DEBUT
Associer chaque variable à son importance
Construire un tableau des importances
Trier les variables par importance décroissante

Afficher le tableau
Construire un graphique en barres
Afficher le graphique

Retourner le tableau des importances
FIN

7.6. Diagramme d'activité de l'arbre de décision régressif

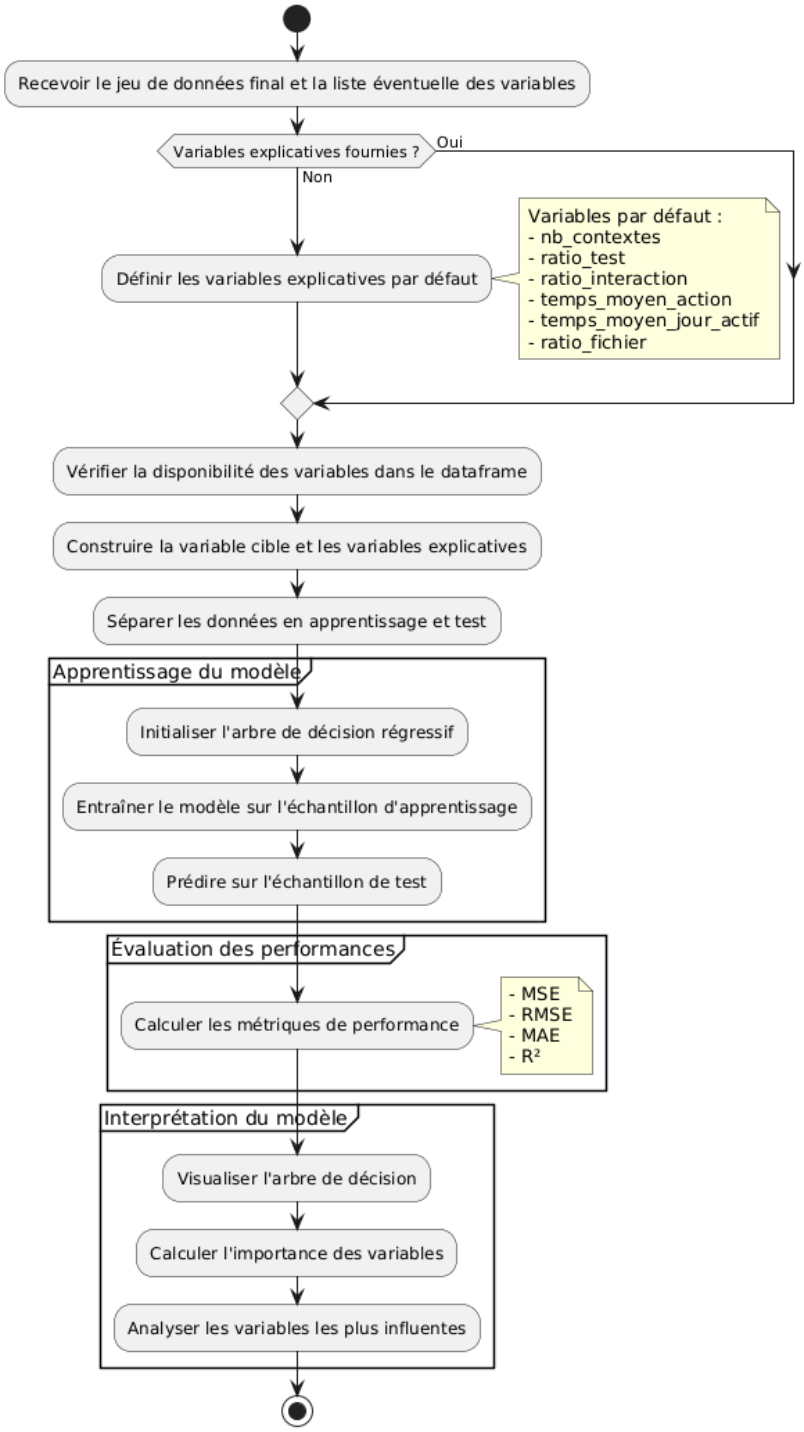


Figure 8 : Diagramme d'activité de l'arbre de décision Régressif

8. Module d'évaluation comparative

Le module *evaluation.py* a pour objectif de comparer les performances des deux modèles construits dans le projet à savoir la régression linéaire multiple et l'arbre de décision régressif. Il ne construit pas de nouveau modèle, mais permet de synthétiser les résultats obtenus afin de faciliter leur comparaison. Le module est structuré en trois étapes :

1) le calcul des métriques ;
2) la comparaison des performances ;
3) la production de visualisations diagnostiques.

8.1. Calcul des métriques

Une fonction dédiée calcule les indicateurs classiques d'évaluation d'un modèle de régression :

- **MSE** : erreur quadratique moyenne ;
- **RMSE** : racine de l'erreur quadratique moyenne ;
- **MAE**: erreur absolue moyenne ;
- **R^2**: coefficient de détermination.

Ces métriques sont calculées à partir des valeurs réelles et des valeurs prédites.

8.2. Comparaison des modèles

Le module calcule les métriques pour chacun des deux modèles, puis construit un tableau comparatif regroupant les résultats.

Une comparaison est ensuite effectuée principalement à partir :

- du RMSE, mesurant l'erreur moyenne de prédiction ;
- et du R^2, mesurant la capacité explicative du modèle.

Cette étape permet d'identifier si un modèle présente de meilleures performances globales ou si les résultats sont contrastés.

8.3. Visualisation des performances

En complément des métriques numériques, le module produit plusieurs visualisations :

- un graphique comparatif des métriques (RMSE, MAE, R^2) sous forme de barres ;
- des graphiques « valeurs réelles vs valeurs prédites » pour chaque modèle ;
- et des distributions des résidus pour analyser la dispersion des erreurs.

Ces visualisations permettent de compléter l'évaluation quantitative en fournissant une lecture graphique du comportement des modèles.

8.4. Sorties du module

Le module retourne un tableau récapitulatif des métriques et les graphiques de comparaison et de diagnostic.

Ces éléments sont utilisés dans l'analyse finale et peuvent être exploités dans l'interface utilisateur.

8.5. Algorithmes de l'évaluation Comparative

Le module d'évaluation a pour rôle de comparer objectivement les performances prédictives des deux modèles retenus dans le projet.

Il calcule plusieurs métriques complémentaires, produit une synthèse tabulaire et graphique, puis formule une conclusion comparative fondée sur la précision et la capacité explicative.

8.5.1. Algorithme principal_evaluation_comparative

Entrées :

- y_test_reg : valeurs réelles du modèle de régression
- y_pred_reg : valeurs prédites par le modèle de régression
- y_test_tree : valeurs réelles du modèle arbre
- y_pred_tree : valeurs prédites par le modèle arbre

Variables :

- metriques_reg : métriques calculées pour la régression multiple
- metriques_tree : métriques calculées pour l'arbre de décision
- resultats : tableau comparatif des performances

Sortie :

- resultats : tableau récapitulatif des métriques des deux modèles

DEBUT
Calculer les métriques de la régression multiple
Calculer les métriques de l'arbre de décision

Construire un tableau comparatif des résultats
Afficher le tableau comparatif

Comparer les performances selon RMSE et R^2

SI l'arbre a un RMSE plus faible ET un R^2 plus élevé
ALORS conclure que l'arbre semble plus performant
SINONSI la régression a un RMSE plus faible ET un R^2 plus élevé
ALORS conclure que la régression semble plus performante
SINON
Conclure que les résultats sont partagés
FINSI

Afficher la comparaison graphique des métriques
Afficher les graphiques valeurs réelles vs valeurs prédites
Afficher les distributions des résidus

Retourner résultats
FIN

8.5.2. Algorithme calculer_metriques(y_true, y_pred)

DEBUT
Calculer l'erreur quadratique moyenne MSE
Calculer la racine de l'erreur quadratique moyenne RMSE
Calculer l'erreur absolue moyenne MAE
Calculer le coefficient de détermination R^2

Retourner les quatre métriques
FIN

8.5.3. Algorithme comparer_modeles(y_test_reg, y_pred_reg, y_test_tree, y_pred_tree)

DEBUT
Calculer les métriques du modèle de régression
Calculer les métriques du modèle arbre

Construire un tableau contenant :
- nom du modèle
- MSE
- RMSE
- MAE
- R^2

Afficher le tableau

Comparer les deux modèles selon RMSE et R^2

SI arbre meilleur sur RMSE et R^2 **ALORS**
Afficher que l'arbre semble plus performant
SINONSI régression meilleure sur RMSE et R^2 **ALORS**
Afficher que la régression semble plus performante
SINON
Afficher que les résultats sont contrastés
FINSI

Retourner le tableau des résultats
FIN

8.5.4. Algorithme afficher_reel_vs_predit(y_true, y_pred, titre)

DEBUT

Construire un nuage de points entre valeurs réelles et valeurs prédites

Déterminer la valeur minimale et la valeur maximale

Tracer la diagonale y = x correspondant à la prédiction parfaite

Ajouter le titre, les axes et la légende

Afficher le graphique

FIN

8.5.5. Algorithme afficher_residus(y_true, y_pred, titre)

DEBUT

Calculer les résidus = valeurs réelles - valeurs prédites
Calculer la moyenne des résidus
Calculer l'écart-type des résidus

Construire l'histogramme des résidus
SI l'écart-type est strictement positif ALORS
  Tracer la courbe d'une loi normale approchée
FINSI

Tracer une ligne verticale au niveau du résidu nul
Tracer une ligne verticale au niveau du résidu moyen

Ajouter le titre, les axes et la légende
Afficher le graphique

FIN

8.5.6. Algorithme afficher_diagnostics_modeles(y_test_reg, y_pred_reg, y_test_tree, y_pred_tree)

DEBUT

Afficher le graphique réel vs prédit pour la régression
Afficher le graphique réel vs prédit pour l'arbre

Afficher la distribution des résidus pour la régression
Afficher la distribution des résidus pour l'arbre

FIN

8.6. Diagramme d'activité de l'évaluation comparative des modèles

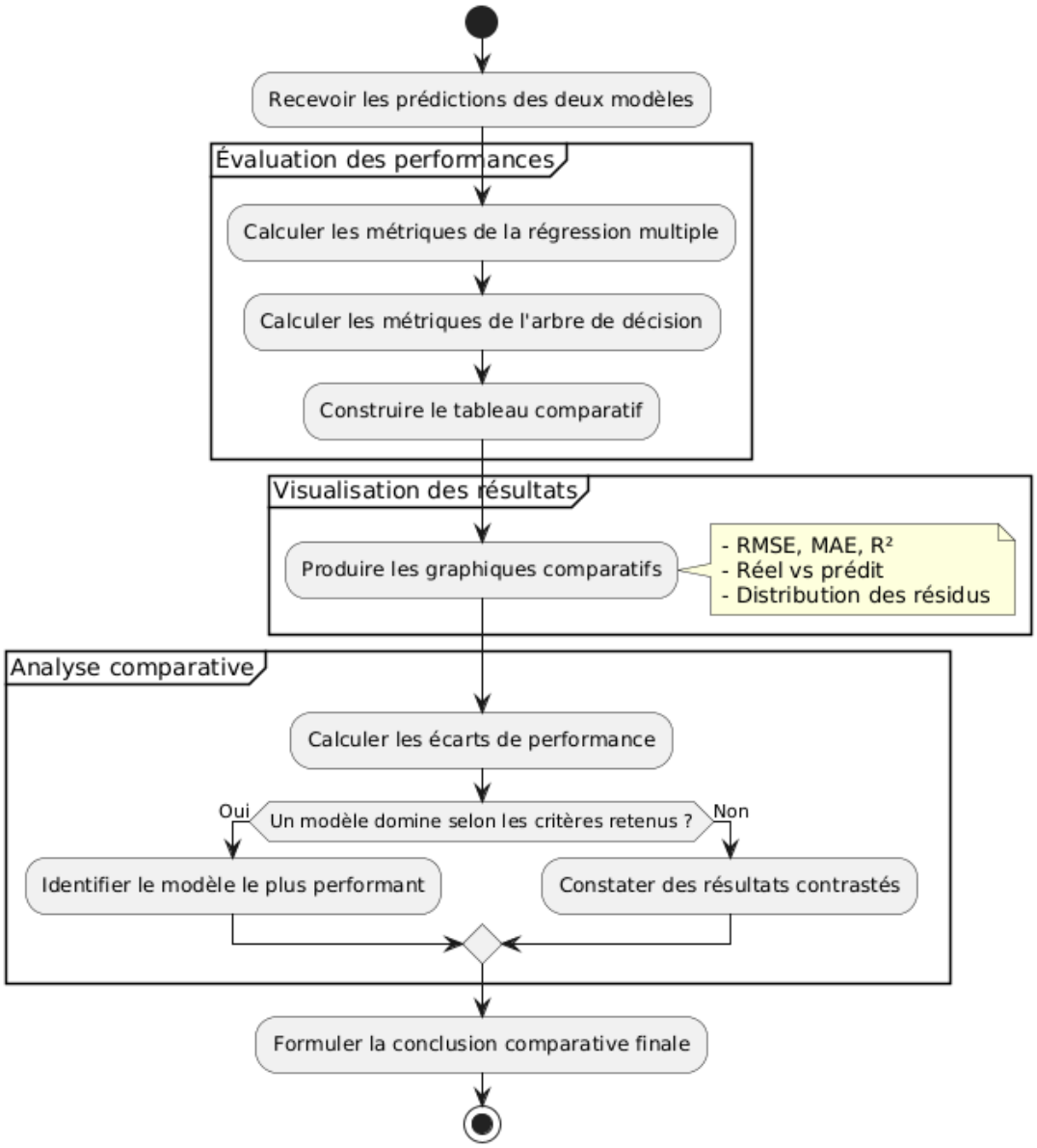


Figure 9 : Diagramme d'activité de l'évaluation comparative des modèles

Ce diagramme décrit la chaîne de comparaison entre la régression multiple et l'arbre de décision, depuis le calcul des métriques jusqu'à la conclusion finale et à l'affichage graphique des résultats.

9. Module Interface Utilisateur

Le module *interface.py* constitue la couche de restitution du projet. Il met à disposition une interface graphique développée avec Tkinter, permettant d'utiliser les modèles construits sans passer directement par le code.

L'interface a pour objectif de permettre à un utilisateur de saisir un profil d'étudiant à partir des variables explicatives retenues et de restituer une prédiction accompagnée d'éléments de lecture synthétiques.

Le module est structuré en quatre étapes :

1) l'initialisation de la fenêtre et des composants graphiques,
2) le chargement et l'entraînement des modèles,
3) la saisie et la validation des variables,
4) et enfin l'affichage des résultats de prédiction.

9.1. Organisation générale

L'application repose sur une classe principale *ApplicationPrediction*, qui hérite de tk.Tk.

Cette classe assure :

- ✓ la création de la fenêtre principale ;
- ✓ la construction des blocs graphiques ;
- ✓ le chargement des données et l'entraînement des modèles au démarrage ;
- ✓ la lecture des entrées utilisateur ;
- ✓ la production des prédictions ;
- ✓ et l'affichage des résultats et des messages associés.

L'interface comprend plusieurs zones :

- un formulaire de saisie ;
- un bloc de résultat principal ;
- un bloc de détail des prédictions ;
- un bloc de lecture du profil ;
- un bloc de repères du jeu de données.

9.2. Initialisation et chargement des modèles

Lors de l'initialisation, l'application exécute automatiquement le pipeline nécessaire à la préparation du moteur de prédiction :

- ✓ chargement des données;
- ✓ prétraitement des fichiers ;
- ✓ construction des variables explicatives;
- ✓ sélection des variables de la régression multiple ;
- ✓ entraînement de la régression multiple ;
- ✓ et entraînement de l'arbre de décision régressif.

L'application calcule également des repères globaux issus du jeu de données, notamment la moyenne du nombre de contextes et la moyenne du ratio d'utilisation des fichiers.

Ces repères sont ensuite affichés dans l'interface pour faciliter la lecture du profil saisi.

9.3. Saisie dynamique des variables

Le formulaire de saisie est construit dynamiquement à partir des variables retenues par la modélisation.

Pour chaque variable explicative sélectionnée, l'interface :

- ✓ crée un libellé explicite,
- ✓ crée un champ de saisie associé,
- ✓ et enregistre ce champ dans une structure permettant de relire facilement les valeurs saisies.

Cette approche permet de maintenir la cohérence entre l'interface et les modèles entraînés.

9.4. Validation des entrées

Avant toute prédiction, l'interface contrôle la validité des valeurs saisies. Les règles de validation dépendent du type de variable :

- les variables de type ratio doivent appartenir à l'intervalle [0,1] ;
- les variables quantitatives comme nb_contextes ou temps_moyen_action doivent être positives.

En cas d'entrée invalide, un message d'erreur est affiché à l'utilisateur.

9.5. Production et affichage des résultats

Une fois les valeurs saisies validées, l'interface :

- ✓ construit le vecteur d'entrée pour la régression multiple
- ✓ construit le vecteur d'entrée pour l'arbre de décision
- ✓ calcule les prédictions des deux modèles

- ✓ affiche les deux notes prédites
- ✓ affiche les métriques globales associées à chaque modèle
- ✓ détermine le modèle recommandé
- ✓ met en avant une note estimée finale

L'interface génère également plusieurs informations synthétiques :

- ✓ un niveau estimé ;
- ✓ un profil simplifié ;
- ✓ une interprétation textuelle ;
- ✓ un positionnement par rapport aux moyennes du dataset ;
- ✓ une recommandation.

9.6. Intérêt du module

L'interface fournit une restitution pédagogique des résultats du projet. Elle ne se limite pas à afficher une valeur de sortie, mais propose une présentation structurée des prédictions et de leur interprétation. Sa finalité est démonstrative parce qu'elle permet d'illustrer concrètement l'utilisation des modèles construits dans le projet, sans constituer pour autant un outil décisionnel opérationnel.

9.7. Algorithmes de l'interface utilisateur

9.7.1. Algorithme principal de l'interface : ApplicationPrediction

Entrée :

- aucune entrée explicite

Variables :

- fenetre : fenêtre principale
- zone_scrollable : zone d'affichage défilante
- widgets : composants graphiques
- variables_reg : variables retenues par la régression
- champs_saisie : champs créés dynamiquement

Sortie :

- interface de prédiction initialisée

DEBUT

Créer la fenêtre principale
Créer la zone scrollable

Entraîner les modèles
Créer les blocs graphiques de l'interface
Créer les widgets de l'interface
Créer les champs de saisie à partir des variables retenues

Retourner une interface prête à être utilisée
FIN

9.7.2. Sous-algorithme : Algorithme _entrainer_modeles

DEBUT
Charger les données
Prétraiter les données
Construire les variables explicatives
Calculer les moyennes du dataset utiles à l'interface
Sélectionner les variables retenues par la régression
Entraîner la régression multiple
Entraîner l'arbre de décision sur les mêmes variables
Construire le texte des repères du jeu de données
FIN

9.7.3. Sous-algorithme : _creer_champs_dynamiques

DEBUT
Initialiser la structure de stockage des champs de saisie

POUR chaque variable retenue
Déterminer le libellé à afficher
Créer un champ de saisie
Associer la variable à son champ
FAIT
FIN

9.7.4. Sous-algorithme : predire_note

DEBUT
Initialiser une structure pour stocker les valeurs saisies
POUR chaque variable de l'interface
Lire la valeur saisie
Vérifier sa validité
Enregistrer la valeur
FAIT

Construire l'entrée du modèle de régression
Construire l'entrée du modèle arbre

Calculer la prédiction de la régression
Calculer la prédiction de l'arbre

Déterminer le modèle recommandé
Afficher la note estimée
Afficher le niveau estimé
Afficher le profil
Afficher l'interprétation
Afficher le positionnement
Afficher la recommandation
FIN

9.7.5. Sous-algorithme : _choisir_modele(pred_reg, pred_tree)

DEBUT
  SI arbre meilleur selon RMSE et R^2 **ALORS**
    Retourner arbre, note arbre, justification
  SINON SI régression meilleure selon RMSE et R^2 **ALORS**
    Retourner régression, note régression, justification
  SINON
    Retourner comparaison nuancée, note régression, justification
  FIN SI
FIN

9.7.6. Sous-algorithme : generer_messages

DEBUT
  Déterminer le niveau estimé
  Classifier le profil
  Générer l'interprétation
  Générer le positionnement
  Générer la recommandation
FIN

9.8. Diagramme d'activité de l'interface

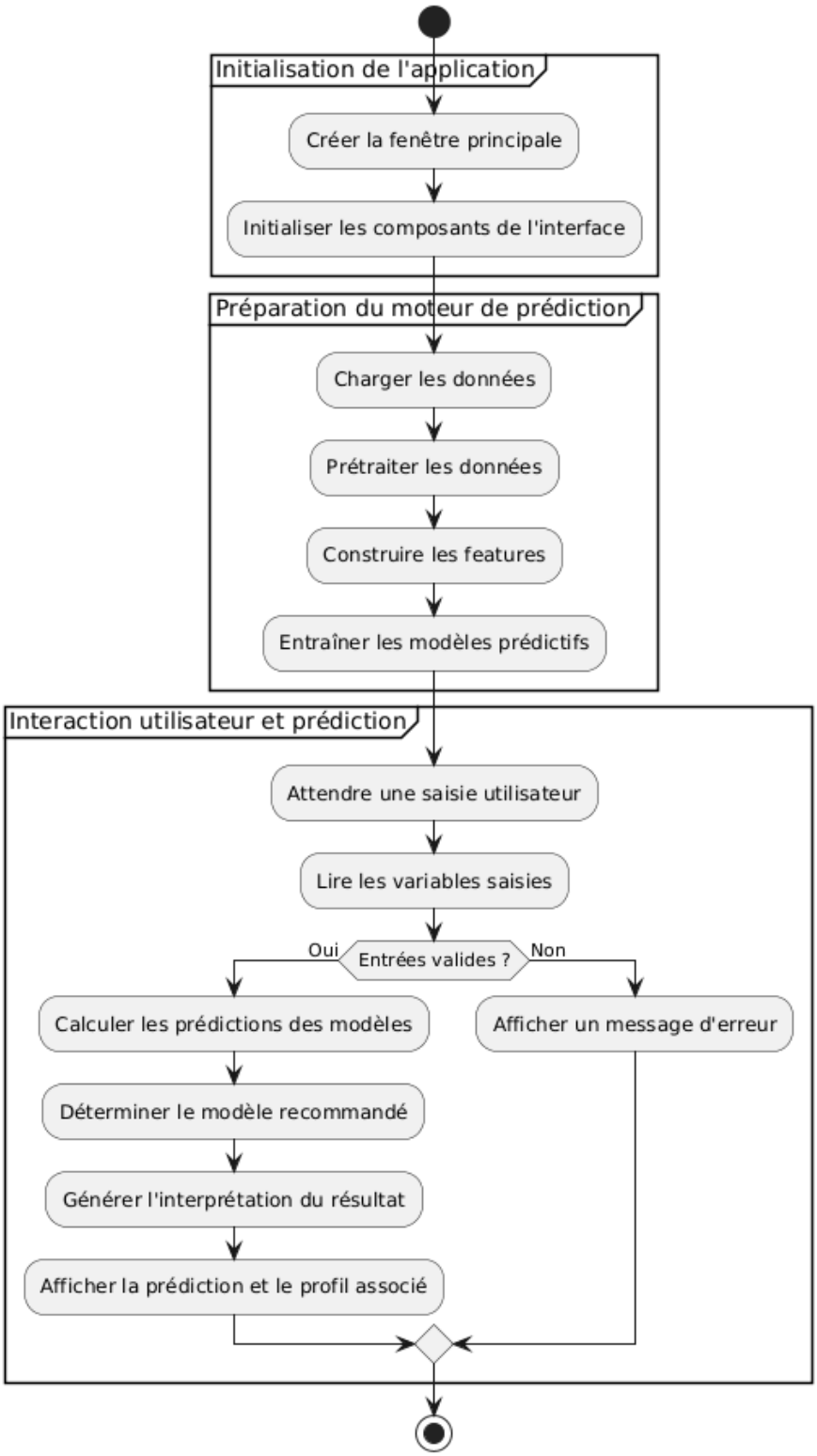


Figure 10 : Diagramme d'activité de l'interface de prédiction

Ce diagramme décrit le fonctionnement général de l'application, depuis l'initialisation de la fenêtre et la préparation du moteur de prédiction jusqu'à la saisie utilisateur, la validation des entrées, le calcul des prédictions et l'affichage des résultats.

Conclusion générale

Ce projet a permis de concevoir et de développer une application complète de prédiction de la note à partir des traces d'activité sur la plateforme ARCHE, en respectant les contraintes du projet en termes d'ETL, de modélisation, d'algorithmique et d'interface utilisateur.

L'architecture du système repose sur une séparation claire des responsabilités, avec des modules dédiés au chargement des données, au prétraitement, à la construction des variables explicatives, à la modélisation, à l'évaluation et à l'interface utilisateur. Cette organisation modulaire permet de garantir la lisibilité, la maintenabilité et la réutilisabilité du code.

Le pipeline de traitement met en œuvre une chaîne cohérente allant des données brutes jusqu'à la restitution des résultats, en assurant la qualité des données et la cohérence entre les différentes étapes. Les choix techniques réalisés, notamment l'utilisation de pandas pour le traitement des données et de scikit-learn pour la modélisation, permettent une implémentation efficace et adaptée au volume de données considéré.

L'intégration d'une interface graphique constitue un élément central du projet, en permettant de relier les modèles développés à un usage concret. L'utilisateur peut ainsi saisir des variables explicatives, obtenir des prédictions issues de plusieurs modèles, et bénéficier d'une interprétation des résultats.

Enfin, ce projet met en évidence l'importance de la structuration d'un système de data science de bout en bout, depuis l'ingénierie des données jusqu'à la mise à disposition d'un outil utilisable. Il constitue une base solide qui pourrait être enrichie par l'intégration de modèles plus avancés, l'ajout de nouvelles sources de données ou l'amélioration des performances prédictives.